Experiment 4: FastAPI-Based ML Inference API Deployment

AIM

To study the development and deployment of a FastAPI-based Machine Learning inference API and compare it with Flask.

OBJECTIVES

1. To understand API-based ML model inference using FastAPI.
2. To compare Flask and FastAPI for ML deployment.
3. To study offline and online deployment using cloud platforms.

THEORY

1. ML Inference using FastAPI

An ML inference API allows users to send input data and receive predictions through HTTP requests.

FastAPI is a high-performance Python framework that supports automatic validation and asynchronous execution. For offline setup, it runs locally using Uvicorn for testing and development.

---

2. Comparison: Flask vs FastAPI

Flask is a lightweight and flexible framework for building APIs. However, it requires additional setup for validation and performance optimization.

FastAPI provides built-in validation, automatic documentation, and better performance, making it more suitable for scalable ML inference systems.

---

3. Deployment with Docker and Cloud Services

Docker is used to package the API with all dependencies for consistent deployment. Containers can be managed using Kubernetes for large-scale applications.

APIs can be deployed on platforms such as:

- Azure App Service
- Cloud Run
- Render

These platforms provide scalable and managed hosting.

---

4. End-to-End Workflow

The model is first tested locally using FastAPI and Uvicorn. After validation, it is containerized and deployed to cloud services for public access.

This approach ensures scalable, reliable, and production-ready ML deployment.

CONCLUSION

In this experiment, we studied FastAPI-based ML inference deployment and compared it with Flask. We also understood local testing and cloud deployment methods.

FastAPI with containerization offers an efficient and scalable solution for ML model serving.